

Javascript. Funciones y Eventos

Currying

Transformar una función que recibe varios argumentos:

```
function sumar(a, b, c) {  
    return a + b + c;  
}
```

Javascript. Funciones y Eventos

Currying

Transformar una función que recibe varios argumentos:

```
function sumar(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    };  
  };  
}
```

```
console.log(sumar(10)(20)(30));
```

Javascript. Funciones y Eventos

Aplicación parcial

Relacionado con currying, pero conceptualmente diferente.

```
function multiplicar(a, b, c) {  
    return a * b * c;  
}
```

```
const multiplicarPor10 = multiplicar.bind(null, 10);
```

```
console.log(multiplicarPor10(2, 3));
```

Javascript. Funciones y Eventos

Curricación

El currying es una técnica donde una función que recibe múltiples argumentos se transforma en una **secuencia de funciones anidadas**, cada una recibiendo un solo argumento. Es muy útil en programación funcional para reutilizar lógica.

```
const sumarCurrying = (a) => (b) => (c) => a + b + c;  
console.log(sumarCurrying(1)(2)(3)); // 6  
const sumarDiez = sumarCurrying(10);  
console.log(sumarDiez(5)(2)); // 17
```

Javascript. Funciones y Eventos

Currying

```
function multiplicar(a, b) {  
    return a * b;  
}
```

Podemos hacer

```
function multiplicar(a) {  
    return function(b) {  
        return a * b;  
    };  
}
```

Javascript. Funciones y Eventos

Currying

Ahora podemos crear funciones especializadas:

```
const duplicar = multiplicar(2);
```

```
const triplicar = multiplicar(3);
```

```
const multiplicarPor10 = multiplicar(10);
```

Javascript. Funciones y Eventos

Funciones generadoras

Una función puede pausar su ejecución y continuar posteriormente.

```
function* numeros() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

```
const generador = numeros();
```

```
console.log(generador.next());  
console.log(generador.next());  
console.log(generador.next());  
console.log(generador.next());
```

Es un concepto muy interesante porque introduce **iteradores y ejecución suspendida**.

Javascript. Funciones y Eventos

Funciones como constructores

Las funciones tradicionales pueden utilizarse con new.

```
function Persona(nombre) {  
    this.nombre = nombre;  
}
```

Esto permite comprender la relación histórica entre:

function->Prototype->new->Constructor->class

```
const p = new Persona("Juan");
```

```
console.log(p.nombre);
```

Javascript. Funciones y Eventos

Prototipo de las funciones

Todas las funciones normales tienen una propiedad prototype.

```
function Persona(nombre) {  
  this.nombre = nombre;  
}
```

P->Persona.prototype->Object.prototype->null

```
Persona.prototype.saludar = function() {  
  console.log(`Hola, soy ${this.nombre}`);  
};
```

```
const p = new Persona("Juan");
```

```
p.saludar();
```

Javascript. Funciones y Eventos

Metaprogramación con funciones

Una función también puede ser manipulada mediante Proxy.

```
function sumar(a, b) {  
    return a + b;  
}  
  
const proxy = new Proxy(sumar, {  
    apply(target, thisArg, argumentos) {  
        console.log("Se llamó a la función");  
        return Reflect.apply(  
            target,  
            thisArg,  
            argumentos  
        );  
    }  
});  
  
console.log(proxy(10, 20));
```

Esto permite interceptar la **invocación de funciones**.

Javascript. Funciones y Eventos

Introspección de funciones

Podemos investigar una función:

```
function sumar(a, b) {  
    return a + b;  
}
```

```
console.log(typeof sumar);  
console.log(sumar.name);  
console.log(sumar.length);  
console.log(sumar.toString());  
console.log(Object.getOwnPropertyNames(sumar));
```

Javascript. Funciones y Eventos

Decoradores de funciones

Podemos crear una función que envuelva otra función para agregarle comportamiento.

```
function registrar(funcion) {  
  return function(...args) {  
    console.log("Argumentos:", args);  
    const resultado = funcion(...args);  
    console.log("Resultado:", resultado);  
    return resultado;  
  };  
}  
  
function sumar(a, b) {  
  return a + b;  
}  
  
const sumarRegistrando = registrar(sumar);  
sumarRegistrando(10, 20);
```

Esto es la base conceptual de muchos mecanismos de:

- logging
- autorización
- caching
- validación
- medición de tiempos
- middleware

Javascript. Funciones y Eventos

Memoización

Una función puede guardar resultados anteriores.

Por ejemplo, podemos utilizarla para evitar cálculos repetidos.

```
function memoizar(funcion) {  
  const cache = new Map();  
  return function(arg) {  
    if (cache.has(arg))  
      return cache.get(arg);  
    const resultado = funcion(arg);  
    cache.set(arg, resultado);  
    return resultado;  
  };  
}
```

Javascript. Funciones y Eventos

Memoización

Por ejemplo, podemos utilizarla para evitar cálculos repetidos.

```
function cuadrado(numero) {  
    console.log("Calculando...");  
    return numero * numero;  
}  
const cuadradoMemoizado = memoizar(cuadrado);  
  
console.log(cuadradoMemoizado(5));  
console.log(cuadradoMemoizado(5));  
console.log(cuadradoMemoizado(5));
```

Javascript. Funciones y Eventos

arguments vs parámetros rest

Las funciones tradicionales tienen acceso al objeto especial:

```
function ejemplo() {  
    console.log(arguments);  
}  
ejemplo(10, 20, 30);
```

O bien

```
function ejemplo(...args) {  
    console.log(args);  
}
```

Javascript. Funciones y Eventos

Arrow functions

```
const sumar = (a, b) => a + b;
```

```
function sumar(a, b) {  
    return a + b;  
}
```

Javascript. Funciones y Eventos

Arrow functions

Por ejemplo, las arrow functions:

no tienen this propio

no tienen arguments propio

no tienen prototype

no pueden utilizarse con new

tienen binding léxico de this

Javascript. Funciones y Eventos

Composición de funciones

```
const duplicar = x => x * 2;  
const sumarUno = x => x + 1;  
const cuadrado = x => x * x;
```

Pasa a ser:

```
const procesar = x =>  
  cuadrado(  
    sumarUno(  
      duplicar(x)  
    )  
  );  
  
console.log(procesar(5));
```

Javascript. Funciones y Eventos

Práctica a4_442. Funciones de orden superior.

Dado un arreglo de objetos con {nombre, clave}, aplicar funciones de primer orden para capitalizar el nombre, y encriptar la clave.

Funciones y Eventos

En el desarrollo web moderno, interactuar con los usuarios a través de **eventos** es fundamental. JavaScript proporciona un poderoso modelo de eventos que permite a los desarrolladores responder a acciones del usuario como clics, movimientos del mouse, pulsaciones de teclas, entre otros.

Acompañando a este modelo, las funciones anónimas ofrecen una manera concisa y conveniente de manejar estos eventos sin necesidad de nombrar explícitamente las funciones

Funciones y Eventos

Fundamentos del Manejo de Eventos

Los eventos son acciones o sucesos que ocurren en el navegador que la página web puede responder, como un usuario

- clickeando un botón,
- llenando un formulario,
- o moviendo el mouse.

Cada uno de estos eventos puede ser detectado y manejado en JavaScript a través de **event listeners** (escuchas de eventos).

Javascript. Funciones y Eventos

Funciones Anónimas

Una función anónima es una función sin un nombre identificador. Son útiles especialmente en contextos donde una función se utiliza temporalmente y no necesita ser referenciada en otro lugar. Son frecuentemente usadas en JavaScript para manejar eventos o realizar operaciones de callback.

```
boton.addEventListener('click', function() {  
  console.log('Botón clickeado!');  
});
```

Javascript. Funciones y Eventos

Agregar un Event Listener:

Utilizando el método `addEventListener`, se puede registrar una función para ser llamada cada vez que un evento específico ocurre en un elemento.

```
const boton = document.getElementById('miBoton');  
boton.addEventListener('click', function() {  
    console.log('Botón clickeado!');  
});
```

Javascript. Manejo del DOM

```
<p id="texto">Texto original</p>
```

```
<button onclick="cambiarTexto()">Cambiar texto</button>
```

```
<script>
```

```
function cambiarTexto() {
```

```
    document.getElementById("texto").innerText = "Texto cambiado";
```

```
}
```

```
</script>
```

Javascript. Manejo del DOM

Creación de elementos

```
<div id="lista"></div>
```

```
<script>
```

```
const lista = document.getElementById("lista");
```

```
// Crear un fragmento (mejor que ir agregando nodo por nodo al DOM directamente)
```

```
const fragment = document.createDocumentFragment();
```

```
for (let i = 1; i <= 1000; i++) {
```

```
    const item = document.createElement("p");
```

```
    item.textContent = `Elemento número ${i}`;
```

```
    fragment.appendChild(item);
```

```
}
```

```
// Insertamos todo de una vez (muy eficiente)
```

```
lista.appendChild(fragment);
```

```
</script>
```

Javascript. Manejo del DOM

Delegación de eventos + modificación de nodos

```
<ul id="tareas">  
  <li>Estudiar JS <button class="eliminar">borrar</button></li>  
  <li>Hacer ejercicio <button class="eliminar">borrar</button></li>  
</ul>
```

```
<script>  
  
const tareas = document.getElementById("tareas");  
  
// Delegación: un solo listener para todos los botones  
tareas.addEventListener("click", e => {  
  if (e.target.classList.contains("eliminar")) {  
    e.target.parentElement.remove(); // elimina el <li> completo  
  }  
});  
</script>
```

Javascript. Manejo del DOM

Modificación masiva con `classList` y atributos

```
<div id="contenedor">
  <p data-role="admin">Usuario: Juan</p>
  <p data-role="user">Usuario: María</p>
</div>
<script>
const usuarios = document.querySelectorAll("#contenedor p");
usuarios.forEach(u => { // Cambiar atributos y clases dinámicamente
  if (u.dataset.role === "admin") {
    u.classList.add("destacado");
    u.setAttribute("title", "Administrador del sistema");
  } else {
    u.classList.add("normal");
    u.removeAttribute("title");
  }
});
</script>
```

```
<style>
.destacado {
  font-weight: bold;
}
.normal {
  color: green;
  font-weight: normal;
}
</style>
```

Javascript. Manejo del DOM

Observadores

Qué problema resuelve Observer?

Supongamos que tenemos un objeto que representa el estado de una aplicación:

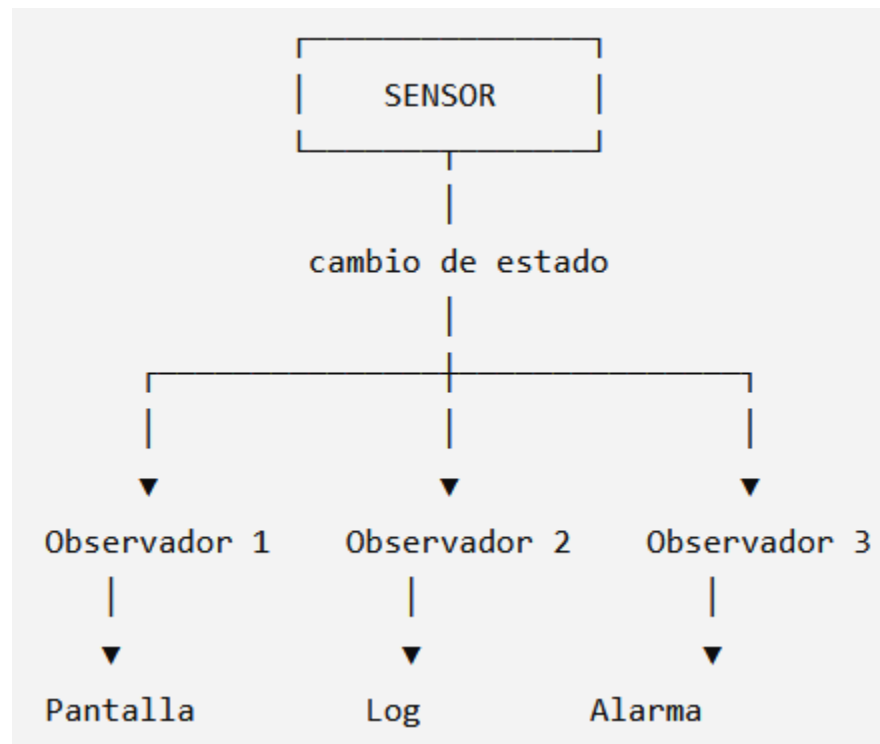
```
const sensor = { temperatura: 20 };
```

Javascript. Manejo del DOM

Observadores

Qué problema resuelve Observer?

Y tenemos varias partes de nuestra aplicación interesadas en saber cuándo cambia la temperatura:



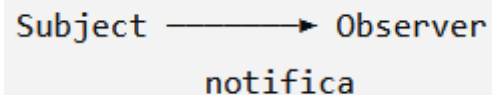
Javascript. Manejo del DOM

Observadores

Qué problema resuelve Observer?

El sensor **no necesita conocer los detalles** de cada uno de ellos. Solamente necesita decir: “Cambió mi estado. Voy a avisar a todos los interesados”. Eso es la idea fundamental del patrón **Observer**.

El patrón Observer establece una relación:



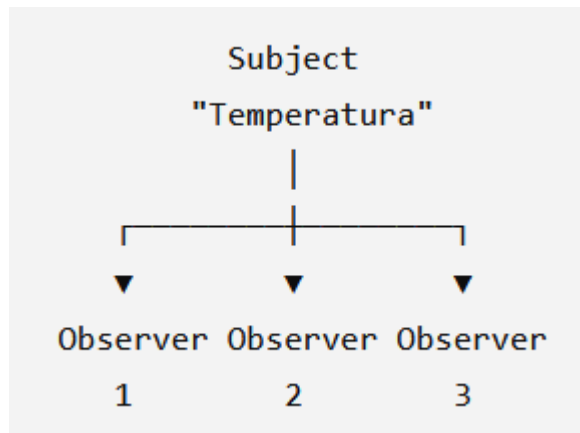
El **Subject** es el objeto que posee información o estado.

Los **Observers** son objetos interesados en recibir notificaciones.

Javascript. Manejo del DOM

Observadores

Por ejemplo:



Cuando cambia el Subject: `subject.notify();`

todos los observers reciben la notificación.

Javascript. Manejo del DOM

Observadores

Por ejemplo:

```
class Subject {  
  constructor() {  
    this.observers = [];  
  }  
  subscribe(observer) {  
    this.observers.push(observer);  
  }  
  notify(data) {  
    this.observers.forEach(observer => {  
      observer(data);  
    });  
  }  
}
```

Javascript. Manejo del DOM

Observadores

Podemos utilizarlo así:

```
const subject = new Subject();  
  
//Ahora agregamos observadores  
  
subject.subscribe(data => {  
  console.log("Observer 1:", data);  
});  
  
subject.subscribe(data => {  
  console.log("Observer 2:", data);  
});
```

Javascript. Manejo del DOM

Observadores

Y notificamos

```
subject.notify("Hola Observers");
```

Resultado:

Observer 1: Hola Observers

Observer 2: Hola Observers

Javascript. Manejo del DOM

Observadores

Implementar Unsubscribe

Nuestro Observer debería permitir que un observador deje de recibir notificaciones.

Podemos agregar:

```
unsubscribe(observer) {  
  
  this.observers =  
    this.observers.filter(o => o !== observer);  
  
}
```

Javascript. Manejo del DOM

[Ver Observer0.html](#)

Observadores

Qué buscamos ?: bajo acoplamiento.

Tener en cuenta que el código de compras podría estar haciendo demasiadas cosas

```
function comprar() {  
    actualizarInventario();  
    enviarEmail();  
    actualizarPantalla();  
    registrarCompra();  
    generarFactura();  
}
```

Con observer

```
function comprar() {  
    compra.notify(producto);  
}
```

Javascript. Manejo del DOM

Observer para cambios en el DOM (MutationObserver)

Los observadores son un patrón esencial en JavaScript, que permite a los desarrolladores **monitorear** y **responder** a los cambios en objetos, eventos o flujos de datos. Proporcionan la columna vertebral para la construcción de aplicaciones dinámicas e interactivas.

Desde el manejo de las interacciones del usuario hasta el monitoreo de flujos de datos complejos, permiten a los desarrolladores crear interfaces receptivas, dinámicas y fáciles de usar.

Javascript. Manejo del DOM

Observadores de eventos (Oyentes de eventos)

Los oyentes de eventos son los observadores más utilizados en JavaScript. Monitorean y responden a eventos como las interacciones del usuario (clics, pulsaciones de teclas o movimientos del ratón).

```
// Seleccionar el boton
const button = document.querySelector('button');

// Asociar un escuchador de eventos para detectar clics.
button.addEventListener('click', (event) => {
  console.log('Button clicked!', event);
});

//Agregar otro escuchador para eventos mouseover
button.addEventListener('mouseover', () => {
  console.log('Mouse is over the button!');
});
```

Javascript. Manejo del DOM

Observadores de mutaciones

Los observadores de mutaciones monitorean los cambios en el DOM, como los nodos añadidos, eliminados o modificados. Esto los hace invaluable para aplicaciones donde el DOM cambia dinámicamente.

```
// Seleccionar el elemento a observar
const targetNode = document.querySelector('#target');

// Crear una instancia de MutationObserver
const observer = new MutationObserver((mutationsList) => {
  mutationsList.forEach((mutation) => {
    console.log('Mutation detected:', mutation);
  });
});

// Opciones de configuración para la observación
const config = {
  childList: true, // para agregar o remover child nodes
  attributes: true, // para cambios en los atributos
  subtree: true, // para cambios en nodos descendientes
};
```

```
// Comenzar a observar el targetNode
observer.observe(targetNode, config);

// Actualizar dinámicamente el DOM
setTimeout(() => {
  const newElement = document.createElement('p');
  newElement.textContent = 'Hello, world!';
  targetNode.appendChild(newElement); // Observer detecta este cambio
}, 2000);
```

Javascript. Manejo del DOM

Observadores de mutaciones

Casos de uso:

- Monitorear actualizaciones de contenido dinámico en aplicaciones de una sola página.
- Detectar cambios de atributos para la configuración de preferencias del usuario (por ejemplo, interruptores de modo oscuro).
- Habilitar funciones de colaboración en tiempo real (por ejemplo, actualizaciones similares a Google Docs).
- Creación de sistemas de notificación en vivo para interfaces de usuario.
- Seguimiento de los cambios en formularios dinámicos o contenido generado por el usuario.

Javascript. Manejo del DOM

Observadores de intersección

Los observadores de intersección monitorean cuando los elementos entran o salen de la ventana. Esto es particularmente útil para imágenes de carga perezosa o activar animaciones.

```
// Seleccionar el elemento a observar
const targetElement = document.querySelector('#box');

// Crear una instancia de IntersectionObserver
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      console.log('Element is in the viewport:', entry.target);
    } else {
      console.log('Element is out of the viewport:', entry.target);
    }
  });
});

// Comenzar la observación del elemento
observer.observe(targetElement);
```

Javascript. Manejo del DOM

Observadores de intersección

Casos de uso:

- Imágenes de carga perezosa para mejorar el rendimiento de la carga de la página.
- Implementación de desplazamiento infinito para sitios web con mucho contenido.
- Activar animaciones CSS o acciones de JavaScript cuando los elementos aparecen en la pantalla.
- Seguimiento y visibilidad para la medición de impresiones.
- Priorizar el contenido por encima del pliegue con fines de SEO. (colocar la información más importante **en la parte de la página que el usuario puede ver sin necesidad de hacer scrol**)

Javascript. Manejo del DOM

Observadores de Redimensionamiento

Los observadores de cambio de tamaño rastrean los cambios en el tamaño de un elemento, lo que los hace ideales para construir interfaces de usuario receptivas.

```
// Seleccionar el elemento a observar
const box = document.querySelector('#resizable');

// Crear una instancia de ResizeObserver
const observer = new ResizeObserver((entries) => {
  entries.forEach((entry) => {
    const { width, height } = entry.contentRect;
    console.log(`New size: ${width}px x ${height}px`);
  });
});
```

```
// Observar el elemento
observer.observe(box);

// Cambiar dinámicamente el tamaño
setTimeout(() => {
  box.style.width = '400px';
  box.style.height = '200px';
}, 2000);
```

Javascript. Manejo del DOM

Observadores de Redimensionamiento

Casos de uso:

- Adaptación de componentes a cambios de tamaño dinámico en diseños responsive.
- Cambiar automáticamente el tamaño de los elementos de la tabla o del lienzo.
- Aplicar dinámicamente consultas basadas en las dimensiones de los elementos.
- Monitorear paneles de cambio de tamaño en un diseño de vista dividida.
- Ajuste de los diseños para las características de personalización del usuario.

Javascript. Manejo del DOM

Observadores de propiedades de objetos

Los cambios de propiedad de objeto se pueden observar utilizando la API Proxy, que permite interceptar y reaccionar a las actualizaciones de propiedades de forma dinámica.

```
// Crear un objeto para observar
const obj = { name: 'John', age: 30 };

// Usar un proxy para interceptar los cambios
const observedObj = new Proxy(obj, {
  set(target, property, value) {
    console.log(`Property ${property} changed from ${target[property]} to ${value}`);
    target[property] = value; // Set the new value
    return true;
  },
});

// Cambiar propiedades para lanzar el observer
observedObj.name = 'Doe'; // Logs: Property name changed from John to Doe
observedObj.age = 31;    // Logs: Property age changed from 30 to 31
```

Javascript. Manejo del DOM

Observadores de propiedades de objetos

Casos de uso:

- Observar y validar cambios en los objetos de estado de aplicación.
- Actualizaciones de propiedades de depuración en sistemas de gestión de estado complejos.
- Restringir actualizaciones no autorizadas a datos confidenciales.
- Creación de modelos de formulario reactivos para la vinculación de datos.
- Implementación de la lógica de validación dinámica basada en los cambios de propiedad.

Javascript. Manejo del DOM

Patrones observables (RxJS) (Reactive Extensions for JavaScript)

RxJS proporciona una implementación robusta del patrón de observación, lo que le permite crear y administrar flujos de datos de manera eficiente.

```
import { Observable } from 'rxjs';
```

```
// Crear un observable
```

```
const observable = new Observable((subscriber) => {  
  subscriber.next('First value');  
  setTimeout(() => subscriber.next('Second value'), 1000);  
  setTimeout(() => subscriber.complete(), 2000); // End the stream  
});
```

```
// Suscribirse al observable
```

```
observable.subscribe({  
  next: (value) => console.log('Received:', value),  
  complete: () => console.log('Stream complete'),  
});
```

Javascript. Manejo del DOM

Patrones observables (RxJS) (Reactive Extensions for JavaScript)

Casos de uso:

- Gestión de flujos de datos asíncronos como solicitudes HTTP o eventos WebSocket.
- Manejar actualizaciones en tiempo real en paneles o notificaciones.
- Implementación de patrones de programación reactivos en marcos front-end.
- Coordinar múltiples operaciones asíncronas con facilidad.
- Construir utilidades reutilizables para la ingestión o limitación de eventos.

Javascript. Manejo del DOM

Observadores de rendimiento

Los observadores de rendimiento supervisan los eventos relacionados con el rendimiento, como la carga de recursos o las tareas largas, para optimizar el rendimiento de la aplicación.

```
// Crear una instancia de PerformanceObserver
const observer = new PerformanceObserver((list) => {
  const entries = list.getEntries();
  entries.forEach((entry) => {
    console.log('Performance entry:', entry);
  });
});

// Comenzar a observar las entradas de temporización de recursos
observer.observe({ entryTypes: ['resource'] });
```

Javascript. Manejo del DOM

Observadores de rendimiento

Casos de uso:

- Depuración y análisis de problemas de rendimiento web.
- Medición de los tiempos de carga de recursos para la optimización.
- Identificar tareas largas que bloqueen el hilo principal.
- Monitorear métricas de experiencia de usuario como Time to Interactive (TTI).
- Analizar el impacto en el rendimiento de los scripts de terceros.

Javascript. Manejo del DOM

Observadores Personalizados

Se pueden construir observadores personalizados utilizando el patrón de diseño de observadores para escenarios no cubiertos por APIs integradas.

```
class Observable {  
  constructor() {  
    this.observers = [];  
  }  
  
  // Suscribirse al observable  
  subscribe(observer) {  
    this.observers.push(observer);  
  }  
  
  // Notificar a todos los observers  
  notify(data) {  
    this.observers.forEach((observer) => observer(data));  
  }  
}  
  
// Crear una instancia de observable  
const observable = new Observable();
```

```
// Suscribirse a notificaciones  
observable.subscribe((data) => console.log('Observer 1:', data));  
observable.subscribe((data) => console.log('Observer 2:', data));  
  
// Notificar a observers  
observable.notify('Hello, Observers!');
```

Javascript. Manejo del DOM

Observadores Personalizados

Casos de uso:

- Creación de sistemas de eventos personalizados para aplicaciones modulares.
- Implementación de arquitecturas reactivas para la gestión del estado.
- Simplificar los modelos pub-sub para componentes desacoplados.
- Apoyo a los mecanismos de notificación específicos de la aplicación.
- Creación de flujos de trabajo dinámicos para acciones impulsadas por el usuario

Javascript. Manejo del DOM

Edición en vivo estilo "Google Docs" (contentEditable)

```
<div id="editor" contenteditable="true" style="border:1px solid black; padding:10px;">
```

Escribe aquí...

```
</div>
```

```
<script>
```

```
const editor = document.getElementById("editor");
```

```
editor.addEventListener("input", () => {
```

```
  console.log("Contenido actual:", editor.innerHTML);
```

```
});
```

```
</script>
```

Javascript. Manejo del DOM

Práctica Observers1 en la plataforma

Javascript. Manejo del DOM

Práctica a4_443

- Crear una página que lleve cuenta de una Todo List como la de la figura.
- Al presionar el botón agregar se renderiza el nuevo ítem Todo y se actualiza el registro de cambios llevado por un observer.

To-Do List

Ir a la facultad

☒ ☐

Hacer las compras

☒ ☐

Registro de cambios en el DOM:

Esperando cambios en el DOM...

Cambio: childList en UL

Cambio: childList en UL

Cambio: attributes en LI